



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2026 - HM



Chapitre 7 : La programmation générique

1. Introduction à la programmation générique
2. Les Classes et Interfaces génériques
3. Les méthodes génériques
4. Les tableaux génériques
5. Les Types Bornés en Java (Bounded Types)

Ensicaen©2025 - HM

2

I. Introduction à la programmation générique

- La programmation générique en Java permet de définir des classes, des interfaces ou des méthodes paramétrées par un ou plusieurs types, sans avoir à définir explicitement ces types à l'avance.
- ➔ Une seule classe générique peut gérer plusieurs types au lieu de dupliquer le code.
- Avec les génériques, le type est défini dès la déclaration, évitant les erreurs de *cast*.
- Les erreurs de type sont détectées à la compilation plutôt qu'à l'exécution, rendant le code plus robuste.

II. Classes génériques

- Une **classe générique** permet de définir un type paramétré **<T>**, qui peut être utilisé comme type de données pour ses attributs et méthodes.

```
class Data<T> { // T : Type paramétré
    private T ob;
    public void setOb(T o) {this.ob = o; }
    public T getOb() { return this.ob; }
}
```

- ➔ On peut créer des objets de la classe Data en précisant le type générique souhaité.
 - `Data<Float> df = new Data<Float>();`
 - `Data<Long> dl = new Data();`
- Le type générique doit **obligatoirement** être une classe enveloppante (**wrapper class**) et non un type primitif.
 - `class Data<int>` : ne sera pas acceptée
 - `class Data<Integer>` : est acceptée « wrapper class : encapsule une valeur int »

II. Classes génériques

- On peut définir une classe générique à **plusieurs paramètres**

```
class DataPP <T, V> {
    private T a;
    private V b;
    public DataPP(T a, V b) {
        this.a = a; this.b = b;
    }
    public void show() {
        System.out.println("Premier: "+ a +" Deuxième: "+ b);
    }
}

public class TestDataPP{
    public static void main(String [] pm) {
        DataPP<Integer, Float> dif = new DataPP<Integer, Float> (5, 2.3F);
        DataPP<String, Character> dbs = new DataPP<String, Character>("Nice", 'A');
        DataPP<Double, Byte> ddb = new DataPP<Double, Byte>(16.8, (byte) 7);
        dif.show();
        dbs.show();
        ddb.show();
    }
}
```

III. Méthodes génériques

- Une **méthode générique** est une méthode qui fonctionne avec des types paramétrés pour être réutilisable avec différents types de données.

```
class MyMethodGen {
    public static <T> void affiche(T pm) {
        System.out.println("paramètre: " +pm);
    }
}

public class TestMyMethodGen {
    public static void main(String[] args) {
        MyMethodGen.affiche("Bonjour");
        MyMethodGen.affiche(7);
        MyMethodGen.affiche('B');
        MyMethodGen.affiche(2.5);
    }
}
```

- <T> : Doit être placé avant le type de retour, indique que la méthode va utiliser un paramètre de type générique,
- pm : argument de la méthode, qui peut être de n'importe quel type.

III. Méthodes génériques

- On peut définir une méthode générique à plusieurs paramètres.

```
class MyMethodGenPP {
    public static <T,E> void display(T a, E b) {
        System.out.println("Premier: " + a + " Deuxième: " + b);
    }
}
public class TestMyMethodGen {
    public static void main(String[] args) {
        MyMethodGenPP.display('A', 65);
        MyMethodGenPP.display("Salut", 2025.3);
    }
}
```

IV. Tableaux génériques

- En Java, les tableaux génériques ne sont pas directement supportés.
- Pour créer un tableau d'un type générique, il faut créer un tableau d'Object et le caster en un tableau du type générique.

```
class MyArray<T> {
    private T A[] =(T[]) new Object[10];
    int size = 0;
    public void append(T x) { A[size++] = x; }
    public void display() {
        for(int i=0; i<size; i++) {
            System.out.println(A[i]);
        }
    }
}
public class TestMyArray {
    public static void main(String[] args) {
        MyArray<Integer> ma = new MyArray<Integer>();
        ma.append(3); ma.append(5); ma.append(9); ma.display();
        MyArray<String> ms = new MyArray<String>();
        ms.append("Hi"); ms.append("Nice"); ms.append("Bye"); ms.display();
    }
}
```

V. Les Types Bornés (Bounded Types)

- Les types bornés en Java permet de restreindre (limiter) les types pouvant être utilisés comme argument de type dans une classe ou méthode générique.

Types bornés supérieurs

Syntaxe : `<T extends Classe>`

- Le mot-clé **extends** permet d'imposer une contrainte sur le type générique. Il permet de limiter le paramètre de type à une classe/interface spécifique ou à ces dérivées.

Exemple :

- `<T extends Number>` signifie que T doit être égale :
 - Soit la classe Number.
 - Soit une sous-classe de Number (Integer, Double, Byte,...)

V. Les Types Bornés (Bounded Types)

Le type String n'est pas une sous classe de **Number**, ce qui provoque une erreur de compilation.

```
class TableauBounded <T extends Number>{
    private T A[] =(T[]) new Number[10];
    int size = 0;
    public void append(T x) { A[size++] = x; }
    public void display() {
        for(int i=0; i<size; i++) {
            System.out.println(A[i]);
        }
    }
}

public class TestTableauBounded {
    public static void main(String[] args) {
        TableauBounded<Double> tb = new TableauBounded();
        tb.append(15.6); tb.append(7.4); tb.append(14.8);
        tb.display();
        TableauBounded<Integer> tb2 = new TableauBounded();
        tb2.append(15); tb2.append(7); tb2.append(14);
        tb2.display();
        TableauBounded<String> tb3 = new TableauBounded();
    }
}
```